Advances in Cognitive Systems 13 (2026) 1-18 Submitted 8/2026; published 10/2026

# Using Metareasoning for Strategy Guidance during Troubleshooting

**James R. Kirk** JAMES.KIRK@CIC.IQMRI.ORG
**Steven J. Jones** STEVEN.JONES@CIC.IQMRI.ORG
Center for Integrated Cognition, IQMRI, Ann Arbor, MI 48105 USA

## Abstract

Troubleshooting, the ability to identify and repair faults, is an essential capability of intelligent agents. Existing SoA computational approaches focus on single strategies selected to match the fault class when it is known a priori. To address novel faults, when the class is unknown beforehand, an agent must consider multiple strategies, but selecting the right one is challenging given the unknowns. Inspired by human troubleshooters, we look to metareasoning (reasoning about reasoning) to guide strategy selection. We present an approach to multi-strategy troubleshooting within a cognitive architecture that integrates multiple troubleshooting strategies and implements appraisals to facilitate the metareasoning needed to inform strategy selection and change. We describe the theory of our approach and provide details of the implementation of new metacognitive state appraisals, troubleshooting strategies, and metareasoning. Our initial results show that the approach efficiently solves various novel fault scenarios in the Factorio factory simulation game by using multiple strategies and switching between them when the agent fails to make progress.

## 1. Introduction

Troubleshooting is a necessary capability for an autonomous agent acting in open worlds. Even the most meticulously engineered systems, crafted with careful foresight and planning, will inevitably encounter behavioral anomalies that require debugging and finding solutions. State-of-the-art computational approaches to troubleshooting generally focus on a single strategy apt for the troubleshooting domain. That is, the troubleshooting approach is chosen based on designer knowledge about the class of faults that can occur in the target domain, enabling selection of the appropriate troubleshooting strategy in design rather than an agent choice in deployment.

We aspire to develop general troubleshooting capabilities to address novel faults (not known a priori) in an agent. By requiring that an agent troubleshoot novel faults (and to do so without complete knowledge of how faults manifest), it is not (generally) possible to choose, at design time, a single appropriate strategy that is both effective and efficient for any novel fault. Instead, the agent needs to have multiple strategies available and the ability to select among them. However, given limited assumed prior knowledge of specific faults, how can an agent select the most appropriate/amenable strategy when first encountering a novel fault? The unknown nature and unpredictable effects of such faults make them difficult to categorize and evaluate. Faults are often tricky to localize and can arise from complex interactions among multiple systems.

© 2026 Cognitive Systems Foundation. All rights reserved.

To address these multiple-strategy challenges, rather than having the agent directly evaluate the fault scenario to determine strategy, we describe an approach in which an agent evaluates its own reasoning (also known as metareasoning) during troubleshooting to guide strategy selection. We take inspiration from humans who excel at troubleshooting novel problems. Humans consider and employ many different strategies, drawing on a wide range of skills and knowledge. Metareasoning (reasoning about reasoning) enables humans to recognize when they are stuck, consider alternative paths, and change strategies. By enabling agent metareasoning, we can shift the problem from the agent directly evaluating the fault (which it does not yet understand) to the agent evaluating itself.

We posit that metareasoning is an effective method for strategy selection when unknowns (novelty) make it difficult to evaluate a strategy directly, and that it can improve troubleshooting efficiency. We also observe that metareasoning requires signals that it should be employed, preferably with an informative trigger, and direct access to agent knowledge used in reasoning. Thus, a metareasoning approach requires some realization of these triggers and access to agent (task) knowledge.

This paper explores this claim and observations, culminating in a functional demonstration of strategy selection and strategy switching in complex troubleshooting scenarios. We first survey and discuss the data and processes required for troubleshooting approaches, which include a wide range of strategies, and map out a multi-dimensional design space. To ground the subsequent discussion of our approach and implementation, we introduce the environment used for evaluation, the engineering-simulation game Factorio. Next, we discuss our approach to integrating multiple troubleshooting strategies into a single agent, which we designed to enable the exploration of metareasoning for guiding strategy selection.

Then, we present an agent, developed within the Soar cognitive architecture, that implements 1) multiple troubleshooting strategies, 2) metacognitive appraisals that trigger metareasoning, and 3) agent ability to switch between strategies during troubleshooting. We illustrate the agent's capabilities in Factorio. We demonstrate its ability to diagnose and solve problems by selecting and switching strategies conditioned on available data and metareasoning.

## 2. Computational Troubleshooting

To understand what support for multiple strategies entails, we first review computational approaches to troubleshooting. Classic model-based reasoning (MBR) frameworks, as surveyed by Davis & Hamscher (1988), establish that while troubleshooting techniques vary in the specific knowledge they leverage, they share an underlying procedural paradigm. This paradigm relies on a formal definition of diagnosis from first principles (Reiter, 1987), typically operationalized via a sequential diagnostic cycle (De Kleer & Williams, 1987). This paradigm operates as a three-stage cycle: (1) *hypothesis generation*, which constructs explanations for discrepancies between observed behavior and expected nominal states; (2) *hypothesis testing*, which uses observations or simulated outcomes to rule out candidate components; and (3) *hypothesis discrimination*, which designs or schedules targeted tests to isolate the true root cause among surviving candidates.

Although troubleshooting systems do not always explicitly execute this cycle sequentially, they generally implement this paradigm. However, they differ in underlying data models and processing

engines. For background on the data and processing of these troubleshooting methods, we elaborate on primary archetypes for such systems, as identified by Venkatasubramanian et al. (2003).

**Process-History and Statistical Approaches:** Process-history or statistical methods rely on historical telemetry records rather than explicit physical or causal models. They model via statistical weights, covariance matrices, alarm distributions, or neural network parameters compiled offline from nominal and faulty system outputs. Online, these models compute pattern classification, dimensionality reduction (e.g., Principal Component Analysis), or statistical distance evaluations. In practice, these data-driven approaches are widely applied to fault detection and classification in complex, high-dimensional domains such as chemical plants, bypassing structural engineering models but increasing reliance on process history data.

**Qualitative Causal and Functional Models:** Qualitative models (Forbus, 2008; Grantham & Ungar, 1990) represent the system's underlying structure and behavior abstractly in terms of discrete relations. This class includes qualitative causal graphs, Multilevel Flow Modeling, Signed Directed Graphs, and Fault Tree Analysis. The required data consists of discrete directed graphs, causal matrices, or rule bases compiled a priori from schematics, engineering manuals, or experts. Processing executes via discrete relational inference, utilizing graph traversal, backward or forward logical chaining, and path-consistency checks to propagate fault states and isolate candidate components.

**Analytical Quantitative Models:** Analytical quantitative methods represent the system via continuous, first-principles physics equations, state-space matrices, or transfer functions. These models are constructed a priori from thermodynamic, electrical, or fluid-dynamic principles, and are driven online by real-time sensor telemetry streams. Processing requires continuous numerical integration, matrix inversion, state estimation (such as Kalman filtering), and analytical residual computation to detect behavioral deviations when continuous dynamics are well characterized.

### 2.1 The Design Space

We generalize these archetypes, treating them as specific extremes of how to reify troubleshooting knowledge within a broad multidimensional design space of computational systems that generate and test troubleshooting hypotheses. We group their attributes into three axes, Representation, Acquisition, and Processing, defining the troubleshooting design space shown in Figure 1.

From this perspective, each archetypal computational approach can be framed as a cluster around a point in design space, with each point corresponding to a single design assumption for representation, processing, and a priori data acquisition. In contrast, many modern troubleshooting systems hybridize and include multiple representations and processing, or combinations outside of the historical archetypes. Nonetheless, these dimensions appear to span the contents of those methods. For example, hierarchical Graph Neural Networks learn from telemetry to construct causal graphs, which then guide network walk algorithms to localize root causes (Wang et al., 2023). Physics-Informed Neural Networks and latent differential equation models use both static a priori physical models and dynamic, in-situ telemetry for troubleshooting by embedding differential constraint equations into deep neural network loss functions (Li et al., 2025; Amin et al., 2026).

Regardless, SoA computational approaches are typically each still a single static configuration in the design space. This is sensible when strategy selection can be done at design time. But, this design-time decision assumes prior knowledge of specific fault classes for their application domain

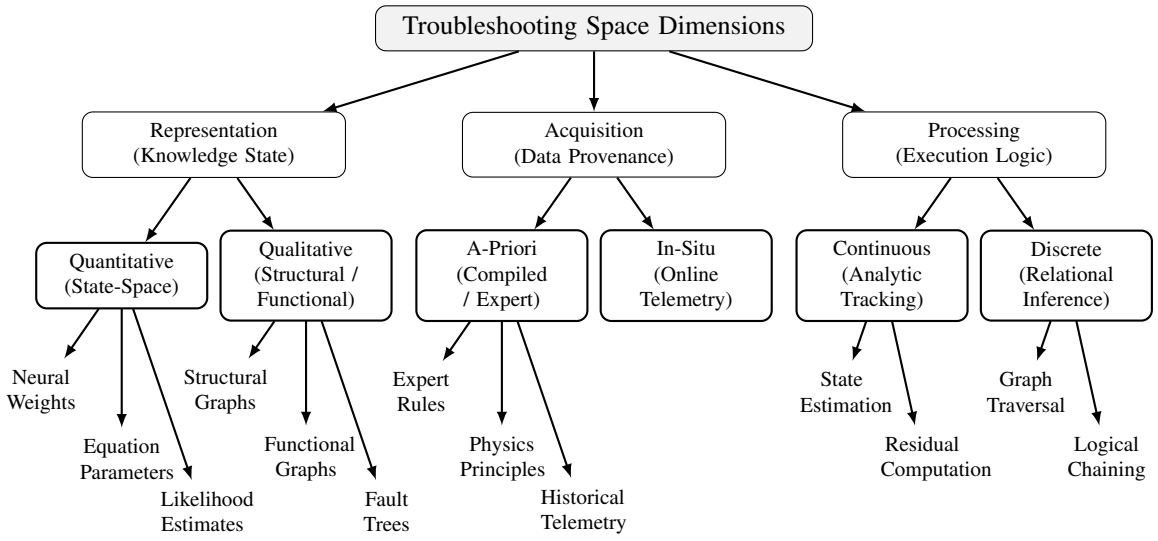



*Figure 1.* Dimensions characterizing diagnostic troubleshooting strategies. Leaves (outside of boxes) are example instantiations among troubleshooting methods.

or large amounts of training data, preventing application to novel or out-of-distribution faults. In contrast, to address novel faults encountered during unexpected situations, multiple strategies must be supported, and selection must occur at runtime. The large space of possible strategies implied by Figure 1 emphasizes the challenge of selecting which strategy is best or even applicable.

## 3. Problem Domain

We use Factorio (Wube Software Ltd., 2020) as the environment in which to realize our multi-strategy troubleshooting approach and evaluate our implementation. Factorio is a real-time engineering simulation game for building, automating, and maintaining massive, complex industrial factories. Factorio can model complex systems and a wide variety of faults that require sophisticated troubleshooting. Causal and spatial relations among components govern dynamics, where complex factories lead to complex fluctuations in resource flows.

There are a large number of different types of components, including power supplies (e.g., steam engines, batteries), transportation tools (e.g., belts, pipes, inserters), forging devices (e.g., assemblers), power-consuming components (e.g., radars), and mining equipment (e.g., electric drills). This creates many resources to track, including power, water, steam, coal, and other materials. It has the desirable characteristic that small local faults can cascade into global system-level failures.

We also selected Factorio because of the Factorio Learning Environment (Hopkins et al., 2026), which was designed to test AI agents. FLE provides API access to observations of components, inventory, flows, and game state and accepts actions as Python code. Agents can repair faults by rotating objects, connecting components, removing components, etc. We forked FLE to expose a more complete game interface, including information about connections between components.

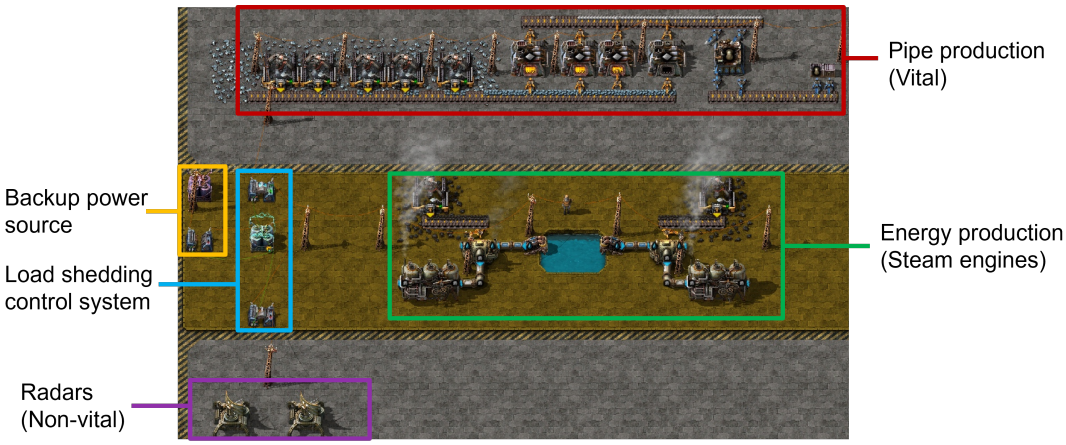



*Figure 2.* Power Load Shedding Scenario with vital (pipe production) and non-vital (radar) systems.

We have developed many different factories in Factorio to serve as troubleshooting scenarios (but focus on one here). We built working versions of each factory and create system models while they run, using a tool that waits for the system to stabilize and measures resource flows to create expectations (average, min, max) for each component. Motivated by a recent, real-world fault, the Francis Scott Key Bridge maritime incident (Ghasemi et al., 2025), we developed a load-shedding scenario for evaluation that shares aspects of that system. "Load shedding" refers to disconnecting non-essential systems. It is typically triggered by a Power Management System that manages systems with designated priority tiers (e.g., non-vital, semi-vital, vital). Load shedding is intended to ensure, when there is an issue (such as power loss), that vital systems maintain power. Load-shedding systems often include backup power generators that are activated when load shedding is triggered to maintain power to vital systems. While extremely useful, load-shedding systems further complicate overall systems management. Further, because they are typically employed infrequently, human users are less familiar with system properties under load-shedding conditions. Thus, faults that arise when load-shedding systems are engaged are often particularly difficult to troubleshoot.

A load-shedding scenario in Factorio is shown in Figure 2. In the middle (green), two steam engines supply power using mined coal and heated water. Circuitry (left middle, blue), acting as the power management system, controls the power (based on availability), supplying the vital system at the top (red, pipe production) and the non-vital system at the bottom (purple, radars). It also controls the backup battery (orange, left middle), which is engaged during a power loss. When operating nominally, the system produces 1.5 pipes per second and fully powers the radars at 600 kW. During load-shedding (e.g., when one of the coal patches runs out, simulating low fuel), with the help of the backup battery, the vital system is maintained (still producing 1.5 pipes per second), while the non-vital radar dishes are left unpowered.

Using this scenario, we developed two distinct fault scenarios (a component fault and a connection fault) for the agent to diagnose, to test the metacognitive appraisals and the ability to switch between strategies.

**Extra Consumption:** In this fault scenario, one assembler is modified so that it consumes more power and iron and produces more pipes per second to simulate a malfunctioning component. The extra power consumption triggers load shedding because there is now not sufficient power for both the vital (pipe production) and non-vital (radar) systems. To repair it, the agent must identify the faulty component and replace it with a new assembler (of the correct version).

**Electrical short during low fuel:** In the second fault scenario, a power pole is added that creates a direct connection from the power production to the radars, simulating an electrical short. In addition, low fuel (empty coal patch) causes the system to attempt load shedding to keep the vital system (pipe production) powered. However, the short prevents proper load shedding, and the non-vital radars continue to draw essential power away from pipe production. To repair the extraneous power, the pole must be removed.

## 4. Approach

The executive layer that enacts multiple strategies must dynamically evaluate available strategies in real time based only on available knowledge. Optimal troubleshooting typically assumes complete prior data or stationary fault distributions (Heckerman et al., 1995), but neither assumption holds given incomplete prior knowledge of faults and their potential complexity. Thus, we pursue an approach that supports one-shot adjustment, satisficing under computational limits, and strategy switching based on incomplete, dynamic knowledge.

Given the breadth of knowledge required, the different processes and data sources among strategies, and the need for online context-sensitive decision-making, we take a cognitive-systems approach, translating computational troubleshooting methods either directly into cognitive structures or into cognitive knowledge of external tool use. By mapping a component strategy directly to an agent's internal cognitive structures, the strategy can be realized as cognitive task and situation knowledge, which enables native metadata and reasoning (metareasoning) relative to that knowledge (Laird et al., 2026). Grounded in introspective monitoring and impasse-driven sub-state reasoning (Cox, 2005; Laird, 2012), an agent can (for example) directly inspect execution traces stored in episodic memory (Cox & Ram, 1999) to evaluate reasoning pathways and diagnose failures. Rather than evaluating the fault scenario directly to determine strategy (difficult, given the potential unknowns), metareasoning can facilitate strategy selection by enabling the agent to evaluate progress in its ongoing reasoning.

A secondary problem is the domain-specific knowledge required to generate and test hypotheses. Given the constraints of novel faults, it can be difficult to know in advance what knowledge might be needed or how to represent it. To address this lack of available prior knowledge, we leverage LLMs as an external tool, endowing a much greater breadth of knowledge to the agent.

We provide implementation details for our system in Section 5, but for the purposes of theoretical alignment, here we generally characterize the methodology of our integration: where troubleshooting strategies are controlled by a single cognitive system, tools are used to acquire domain-specific knowledge, and metacognitive state appraisals that report metadata about the cognitive state during troubleshooting enable adaptation through metareasoning.

### 4.1 Integrating Multiple Troubleshooting Strategies

How can we integrate many computational strategies for troubleshooting? We can define this integration space abstractly by mapping the data representations and processing logic of various troubleshooting strategies (i.e., as in Figure 1) to the structural components of standard cognitive architectures (such as described by the Common Model of Cognition, Laird et al., 2017). The incremental intake from perception matches to in-situ data acquisition, knowledge engineering of an agent before deployment matches to various methods' training over a priori data, the encoding in a specific memory system matches to a method's data format/structure, and cognitive capabilities of an architecture can be used to realize different strategies' processing. Figure 3 details how the dimensions of computational troubleshooting project onto typical core modules of a cognitive architecture.

Direct implementation of a strategy in the agent (providing maximum accessibility to metareasoning) is preferred, but, when necessary given functional limitations, an agent can wrap a strategy as tool usage and maintain knowledge of the tool's performance (Zilberstein, 1996). Thus, usage of the tool remains available to metareasoning, albeit without the same level of knowledge access.

Crucially, external computational methods, such as standalone solvers or optimization engines, are modeled as directly connected to Working Memory via specialized data execution interfaces. Rather than using physical manipulation and sensory feedback, out of engineering practicality, we assume the agent interacts directly with external frameworks via programmatic data structures. Procedural memory executes tool-use skills to populate these memory buffers, invokes the external calculation as an atomic cognitive step, and returns the execution results directly into working memory, thereby keeping the strategy's overall interface accessible to metareasoning.

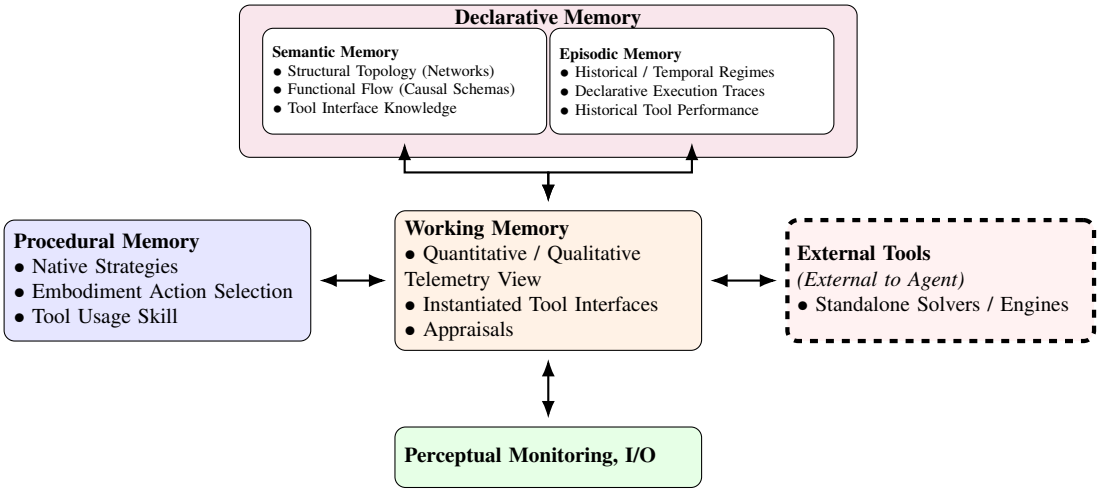



*Figure 3.* Mapping of native troubleshooting mechanisms and external tool integration across common structural components of the cognitive architectures.

### 4.2 Tools for Domain-specific Knowledge Acquisition

Troubleshooting often requires domain-specific knowledge, but with novel fault scenarios, what knowledge may be needed cannot be known in advance. Thus, rather than futilely attempting to import or hand-code all possible needed domain-specific knowledge into the agent, we leverage the breadth of knowledge implicitly available in Large Language Models (LLMs). LLMs are external tools in our implementation (detailed in the next section). We describe their integration methodology here, including how they are used to acquire domain-specific troubleshooting knowledge for hypotheses and repairs, as well as to afford additional filesystem interactions. This access is described via two methodologies: STARS (for directly acquiring domain-specific knowledge) and Agent SDK (for more complex interactions that leverage filesystem knowledge).[1]

#### *4.2.1 STARS*

Building on prior work, we adapted the STARS (Kirk et al., 2024) method for acquiring high-quality, situationally relevant knowledge from an LLM. Using prompts for the desired knowledge, STARS first retrieves a breadth of responses via log-probability beam search (Search Tree). Using this set of responses, it evaluates them to determine which are viable (e.g., understandable by the agent and situationally relevant) and repairs unviable responses via additional prompting (Analysis & Repair). Finally, the agent prompts the LLM one last time to select the best responses from those marked as viable (Selection). We adapt STARS for the troubleshooting domain to generate Factorio-specific hypotheses (given system context) and repairs (given the context of a selected hypothesis).

Modern frontier models no longer support accessing log-probability and/or alternative token generations. To adapt the Search Tree process to work with more recent models, we adopted Verbalized Sampling (Zhang et al., 2025) by adding a prompt instruction to return a set of responses with corresponding probabilities. These probabilities are also used to perform the final stage of selection (selecting the response with the highest probability).

We adapted the Analysis & Repair for this domain to ensure retrieved responses were viable for the agent's domain (in this case, Factorio). The agent analyzes responses to ensure they refer to existing system resources, observed model components, and known repair actions. This allows the agent to test the applicability of generated hypotheses and repairs and reject unviable ones. We have not had to implement any repair mechanisms because STARS successfully generated a high percentage of viable responses in this domain. Using this method, the agent can retrieve a set of viable hypotheses and repairs for the current situation and process them from high to low probability.

#### *4.2.2 Agent SDK*

The second way the agent utilizes an LLM is via Agent SDK (Anthropic, 2026), which enables embedding Claude Code inside an agent architecture. The Soar agent can issue a complex command that results in many iterations by Claude Code. This allows the agent to (indirectly) interact with the file system, issue terminal commands via the CLI, use Claude Code capabilities (e.g., skills, image analysis, MCPs), and maintain a thread of prompts and responses as it seeks to achieve the goals of the instructions. This greatly expands the capabilities of an agent that would otherwise struggle

1. The models used in experiments were respectively GPT 4o and Claude Opus 4.6.

to, for example, search for files, perform actions such as text comparisons, and even run commands like taking screenshots. The specific use of the Agent SDK is detailed further in section 5.2.3.

### 4.3 Using Metacognitive Appraisals to Inform Metareasoning

Metacognitive appraisals can provide signals about the progress (or lack thereof) in an agent's problem solving. Using the context of the agent's problem space, an appraisal can cue metareasoning. For troubleshooting, regardless of whether a strategy is executed entirely natively or via an external tool, these cues can act as feedback signals that inform metareasoning on how to facilitate progress.

While these signals share a functional resemblance to humanlike appraisals, we consider them as functional analogs designed to monitor and characterize the health and constraints of the agent's active search space rather than to model biological affect or humanlike implementation of appraisal generation. They support bounded introspection as described by Cox (2005). Boundedness is realized by efficient computation of metacognitive metadata characterizing the current progress on a problem. This metadata provides the cognitive executive with the necessary signals to enact dynamic strategy-switching. Table 1 details different abstract task-state appraisals and potential metareasoning reactions to address them.[2] The first three entries in the table (impasse, indecision, conflict) have long existed in our implementation architecture, Soar, while the others would be useful additions that, if implemented, could be used to expand Soar's metareasoning capabilities.

*Table 1.* Potential Usage of Appraisal Signals to Inform Metareasoning

| **Metacognitive State Appraisal** | **Potential Metareasoning Reaction** |
|---|---|
| Impasse: no action proposals | Search, learn new proposal... |
| Indecision: uncertain which action to take | Search, heuristics... |
| Conflict: competing preferences | Search, prune option, learn new preference |
| Unconfident: low confidence in search space | Backtrack, abandon problem space |
| Stagnation: too much time without progress | Change goal/strategy selection |
| Overload: too large a branching factor | Backtrack & explore different path, prune |
| Deficit: too few available actions | Gather more data, propose new approach |
| Looping: revisiting states, repeating sequences | Backtrack, abandon current path |
| Runaway: Runaway processing (large substate stack) | Abandon current path |

## 5. Implementation

To explore our approach to multi-strategy troubleshooting and realize it in a cognitive system, we implemented new metacognitive state appraisals, multiple troubleshooting strategies, and metareasoning strategies that react to the appraisals to guide troubleshooting.

We focused on strategies in the qualitative functional modeling archetype, augmented with statistical knowledge mined from process history data (as described in section 2). The troubleshooting strategies are implemented in a Soar agent (providing the representations and the processing for

2. The term "action" in the table can refer to external or internal operations.

fault diagnosis and metareasoning). As outlined above, the Soar agent can also use Large Language Models as an external tool (providing mechanisms for retrieving domain-specific knowledge and accessing the file system). The following sections provide details of these implementations: the metacognitive appraisals, three troubleshooting strategies, and metareasoning.

### 5.1 Metacognitive Appraisals

We implemented three new metacognitive state appraisals in the agent (drawn from Table 1) to evaluate their effectiveness for providing cues for metareasoning during troubleshooting. These appraisals enable the agent to detect whether it is overloaded (too many paths to explore in the current troubleshooting space), stagnating (too much time spent in the current strategy), or unconfident (all options have a low probability of success). The agent uses these appraisals to trigger metareasoning, which may cause it to abandon strategies or hypotheses and switch to others. Table 2 provides the architectural implementation and metareasoning responses for each appraisal.

*Table 2.* Examples of Metacognitive State Appraisals and Responses

| **Appraisal** | **Architecture Implementation** | **MR Reactions during TS** |
|---|---|---|
| Overload | Count of acceptable operator ties above threshold | Switch to other available strategies |
| Unconfident | All remaining option probabilities below threshold | Select other available hypotheses, strategies |
| Stagnation | Elapsed time or count of executed actions above threshold | Select other available hypotheses, strategies |

Overload triggers when the count of tied operators exceeds a predefined threshold. Unconfident triggers when all options in a problem space have probabilities (if available) that are below a specific threshold. Stagnation triggers when the count of executed actions (agent repair attempts) performed in the current problem space exceeds a predefined threshold. Another implementation option for stagnation would be to count decision cycles, operator selections, or clock time. How the agent reacts to these metacognitive appraisals (metareasoning) depends on the agent's current problem space. In some cases, it may not result in any change in the agent's processing.

### 5.2 Troubleshooting Strategies

Drawing on troubleshooting literature, we prototyped three general strategies that the agent can use (based on its available data) to diagnose and repair faults: component crawl (going through each component with any discrepancy from nominal operation[3] and proposing hypotheses and repairs), stock-and-flow causal analysis (using stock-and-flow models for a functional analysis to generate hypotheses and repairs based on system-level resource flow outside expectations), and a structural comparison (where agent SDK is used to compare current structures against data captured during normal operations). The intent is to implement general troubleshooting strategies, not specific to

3. A difference from measured expected rates or a system-reported warning.

the domain of Factorio, and to rely only on the LLM for on-demand domain-specific knowledge. However, we do make some assumptions about the system the agent is troubleshooting for these strategies: that the system reaches a desired equilibrium (possibly with oscillations) when working properly, that faults disrupt this equilibrium, and that repairs can be executed to return the system to equilibrium (and safely undone if they fail).

This initial implementation supports only a small subset of potential troubleshooting knowledge implied by the larger design space outlined in section 2.1. But this subset is sufficient to investigate the challenges of integrating multiple strategies and diverse knowledge. The knowledge representations include cognition-native representations (encoded in semantic memory) of qualitative representations of resource flows and transformation as functional graphs, component connections as structural graphs, and quantitative measurements of different flow rates, retrieved as working memory representations when used to interpret telemetry. Data acquisition is performed by a mix of offline (creating models of the working system) and online (both rote system observation and domain-specific LLM knowledge retrieval) processes. Processing is implemented via production rules (procedural memory) performing graph traversal, logical chaining, and relational inference to analyze the system state, the hypotheses generated, the repairs generated to test those hypotheses, and, finally, the results of attempted repairs.

We provide further details of each strategy below.

### *5.2.1 Component crawl*

The simplest troubleshooting strategy we implemented is a component crawl strategy. The agent steps through each component with a discrepancy (or a reading outside expectations) and proposes hypotheses about what fault in that component could cause that discrepancy. Three hypotheses for this component are generated via the STARS LLM method (described in section 4.2.1) using details of the component as context. The agent selects the highest-probability hypothesis and, using this hypothesis as context, generates 5 repairs to address it again via the STARS method. The agent attempts the highest-probability repair and then reanalyzes the system (after waiting a short time for the system to stabilize). If the fault is repaired and no issues are detected, the hypothesis and repair are marked as correct. Otherwise, the repair is rejected, and the next-highest-probability repair is attempted. If all repairs fail, the current hypothesis is rejected, and the next-highest-probability hypothesis is selected. If all hypotheses are rejected, the agent considers the next component. This strategy is effective when little data is available or when few components have discrepancies.

### *5.2.2 Stock-and-flow functional analysis*

The second strategy is a stock-and-flow (Lane, 2008; Schoenenberger et al., 2021) analysis. We created external (python) tools to create a stock-and-flow functional graph representation of the system that, based on the system layout and data from running the system for a specified window of time, captures the consumers and producers of different resource types, how each component is connected, and the expected quantities and flow rates (average, max, min) for each of these resources. For example, a boiler is a consumer of water (from a water pump) and a producer of steam (to a steam engine). The graph is provided to the troubleshooting agent's semantic memory.

This troubleshooting strategy is proposed when a fault is detected, and the agent has the stock-and-flow model available. Rather than focusing on a single component, the agent observes all deviations detected from the expectations and creates a map of connected components (with their expectations), along with any available information about those components. The agent uses this data as context and, as in the prior strategy, generates a set of hypotheses via the STARS method. Selecting the highest-probability hypotheses, repairs are then generated via STARS. The process repeats as in the above strategy until a correct repair is found or all hypotheses are rejected.

#### *5.2.3 Structural Comparison*

The final strategy we implemented is a structural comparison strategy that looks for differences between the current system and stored data from when the system was working. This strategy would be difficult to implement in a Soar agent because it lacks tools to read and compare files or images. To implement this strategy, we again rely on an LLM, specifically Agent SDK (described in section 4.2.2). The agent provides context of the current system components and connections and instructs Agent SDK to identify what is relevant to the current system, take a snapshot of the current system, and then compare the current data (text and images) against stored data of the working system. Agent SDK is told, through structural comparison, to determine what difference is likely causing the fault and how to repair it. Finally, it is instructed to search for and read documentation files (e.g., .md files) to determine proper Python syntax to perform the repair. Agent SDK is configured to keep iterating (auto-accepting interaction questions) until it discovers the repair. Using the response from Agent SDK, the Soar agent attempts the repair and evaluates whether it was successful.

### 5.3 Metareasoning for Strategy Guidance

The knowledge that controls and operates troubleshooting strategies is represented natively in the agent, making it directly available to metareasoning. Metacognitive state appraisals report evaluations of different aspects of the cognitive state directly in working memory. While these metacognitive appraisals are designed to signal that a change (e.g., switching strategies) may be needed, this may not always be the case (e.g., there are no other possible strategies to explore), so we implement metareasoning as a situation-specific response to metacognitive appraisal cues.

For situation-specific adaptation, metareasoning is implemented in the agent in the same manner as direct troubleshooting reasoning: as procedural memory reasoning in response to the current situation. The distinction is that metareasoning responds specifically to the state of ongoing troubleshooting reasoning. But, otherwise, metareasoning has access to all of the knowledge and context that led the agent down its current reasoning path: the context of its reasoning path (e.g., substate stack), the current problem space, and triggered appraisals to evaluate its current direction and decide what (if any) actions it should take.

When the agent encounters a metacognitive appraisal of overload, it signals that there are too many options to explore efficiently. When it encounters this appraisal while pursuing a troubleshooting strategy and has the necessary data to pursue another, the agent will abandon this strategy and

select the new one. If the agent only has supporting knowledge to pursue the current strategy, it will not abandon it even when the overload appraisal is detected.

When a metacognitive appraisal of stagnation is detected, this indicates that no progress is being made. If the agent is in the midst of pursuing a troubleshooting strategy and others are available, as before, it will abandon the current strategy to pursue others. The agent can return to abandoned strategies if the others prove to be unfruitful.

When the unconfident appraisal is triggered, the agent analyzes the context of the current problem space to determine which actions to take. If the unconfident appraisal is in the repair problem space, indicating it expects none of the generated repairs will successfully address the hypothesized issue, it will abandon this hypothesis and prefer to evaluate others (rather than attempting all the low-probability repairs). If the unconfident appraisal is in the hypothesis problem space, indicating that it expects that none of the hypotheses are correct, then it will abandon the current troubleshooting strategy that led to these hypotheses (assuming other strategies are available to pursue).

Specific examples of the agent using the metacognitive appraisals to perform metareasoning and change strategies are described in section 6.

## 6. Functional Demonstrations

In the following sections, we describe the results of our implementation performing troubleshooting of different load-shedding scenarios (described in Section 3) in Factorio. For this investigation, we have limited the complexity of faults to single-fault scenarios, in which the fault occurs in a component or a connection between components. We describe future plans to evaluate the system with more complex fault situations in section 7.

We evaluate the agent's ability to repair the faults in these scenarios. When the agent has sufficient knowledge to use all three implemented troubleshooting strategies, it progresses (as needed) from component crawl to functional stock-and-flow analysis and then finally to structural analysis.

### 6.1 Extra Consumption

For the first scenario, where an assembler is modified, after the agent detects that the system is in an unexpected load-shedding mode, it begins executing the component crawl strategy. However, some time has passed since the fault, and many components, in addition to the faulty assembler, are outside expected behavior, which triggers the metacognitive appraisal of overload. Through metareasoning, the agent decides to abandon component crawl and switch to stock-and-flow functional analysis. The agent analyzes the stock-and-flow model and creates a map of interconnected components that are outside expectations. Using this as context, the agent, via the STARS method, generates many fault hypotheses. The first hypothesis generated, in JSON, is displayed below:


```
"component": "assembling-machine-2",
"resource": "energy",
"fault-hypothesis": "The assembling machine is overpowered...",
"fault-type": "overpowered",
"probability": 0.85
```

This hypothesis (which happens to be correct) is selected first for evaluation. The agent, again using the adapted STARS method, retrieves a set of repairs from the LLM that might address this hypothesis. The first repair (again correct) is shown below. Executing this repair fixes the fault.

```
"repair-description": "Downgrade the assembling machine to reduce
                       its energy consumption...",
"component": "assembling-machine-2",
"repair": "downgrade",
"python-repair": "move_to(Position(x=44.5, y=16.5));pickup_entity(
                  Prototype.AssemblingMachine3, position=Position(
                  x=44.5,y=16.5));place_entity...",
"probability": 0.85
```

To test the efficacy of metareasoning we repeated this experiment with it disabled. Under that condition, the agent no longer abandons the component crawl strategy, and when it eventually selects the correct component, it successfully generates a correct hypothesis and repair to fix the fault. However, it takes ~30 times longer to find the correct fix, demonstrating the utility of metareasoning in making troubleshooting more efficient.

### 6.2 Electrical short during low fuel

In the second scenario, an electrical short connects power to the low-priority radars while a load-shedding condition is triggered by a lack of fuel (one of the coal patches is exhausted). The agent detects a fault and begins with the component crawl strategy and, as before, hits a metacognitive appraisal of overload, causing it to switch to the causal stock-and-flow strategy.

Using the stock-and-flow analysis as context, the agent generates many hypotheses and proceeds through them, generating and testing repairs. However, all of these are incorrect and fail. The agent's stock-and-flow model for this scenario lacks details about the extraneous power pole, which adds a new consumer/producer pipeline directly from the power supply to the radar system. Without this knowledge, it fails to identify the root cause. After trying several hypotheses and associated repairs, a metacognitive appraisal of stagnation is triggered. Using metareasoning to avoid wasting more time pursuing an unfruitful path, the agent changes to the structural analysis strategy.

Using this third strategy, the agent, via Agent SDK, sends a prompt to compare the current situation against stored data from when it was working. Agent SDK (e.g., Claude Code) first searches system folders for relevant images and text files, then runs a snapshot command to capture an image of the current scenario. Using comparison tools, it compares the current context and image against the working data it found. It then generates a correct hypothesis that there is an extra power pole. A portion of this generation is shown below:

> Looking at the electric poles, the **working model** has a pole at (7.5, 42.5) connected ONLY to the power switch at (7.0, 37.0) — 'connected-electric-entity-id: [813]'. This means when the power switch is disabled (as it should be in L3 Shed mode), the radars below are properly disconnected from power.
>
> The **current system** has an **extra medium-electric-pole at (14.5, 39.5)** (id 816) that does NOT exist in the working model. This pole connects to both: - Pole 791 at (16.5, 31.5) — the **main grid** - Pole 817 at (7.5, 42.5) — the **radar sub-network**

This creates a bypass around the disabled power switch at (7.0, 37.0)...

With this surprisingly in-depth and accurate hypothesis, it can then determine the proper repair action. Agent SDK searches for documentation (e.g., .md files) about action commands to learn the proper syntax and actions to execute. The JSON below is returned to the Soar agent, which uses it to successfully submit the action to FLE. After the action is executed, the agent verifies that the fault no longer exists and that high-priority pipe production is properly supported.

```
"faulty-component": "medium-electric-pole",
"fault": "Extra pole at (14.5, 39.5) bypasses L3 power-switch,
         powering radars that should be shed",
"fault-repair": "pickup_entity(Prototype.MediumElectricPole,
                 Position(x=14.5, y=39.5))"
```

## 7. Conclusions

We have demonstrated how metareasoning can be an effective method for agent strategy selection during troubleshooting when direct knowledge for strategy evaluation is unavailable. Supporting multiple strategies for troubleshooting (and the ability to switch between them dynamically) is crucial when the space of possible faults is unknown and strategies cannot be selected at design time (as in SoA approaches). A long-term, yet unrealized, goal is to build agents that approach the general competency and efficiency of expert human troubleshooters, who excel at using many different strategies and adapting to and addressing unexpected, novel faults.

Our initial results show the promise of our metareasoning approach to multi-strategy troubleshooting developed in a Soar agent. Implementing troubleshooting directly in the agent makes the knowledge and operations used for reasoning available for appraisals and metareasoning. Metacognitive state appraisals (such as overload or stagnation) provide informative signals to trigger metareasoning, and through metareasoning, the agent can evaluate its troubleshooting reasoning to guide strategy selection. In Factorio, in a complex load-shedding scenario, we show the agent abandoning ineffective strategies and switching to more appropriate ones to efficiently identify and repair faults.

We note that our design shares commonalities with several approaches in the SAIL-ON program (e.g. Klenk et al., 2020; Musliner et al., 2021). Our system maintains explicit causal models of domain operation, uses expectation monitoring to detect when operation is unusual, and triggers a principled reasoning process (including hypothesis testing) to recover. However, our work focuses on efficiently addressing unforeseen faults that are still nonetheless expected to be broadly within the span of an agent's existing affordances (with additional understanding or knowledge). Adaptation means restoring previous/target functionality in one shot, not adapting to a new physics. The form of novelty and the challenges faced by the agent are different for the two programs, but we share several techniques, including the overall application of goal reasoning, strategy selection, and model-based understanding, but integrated differently.

In the future, we plan to develop a larger testbed of systems and faults to further test the agent and compare against other approaches (e.g., standalone LLM agents or pure model-based diagnosis). We will expand our testbed of fault scenarios along different characteristic dimensions that

govern the difficulty of fault diagnosis, including the number of simultaneous faults, the causal interaction depth of a fault, and degree of partial observability (Davis, 1984; De Kleer & Williams, 1987; Reiter, 1987). To compare against other approaches, we will evaluate relative cost (solution time, system downtime, number of repairs, estimated cost of repairs) and ability to learn long-term knowledge.

We also plan to further explore metareasoning as a foundational capability of agents that enables self-analysis. The complexity of metareasoning will need to increase as we explore a larger space of possible problem spaces and a larger space of potential actions. For example, new metacognitive appraisals, such as a deficit (too few options), could enable metareasoning that leads the agent to change model representations or acquire more data. In general, we will expand the agent's capabilities by developing additional troubleshooting strategies, exploring a broader range of approaches and representations, and incorporating more metacognitive appraisals.

## Acknowledgements

This work was supported by the Office of Naval Research, contract N00014-25-1-2356. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Department of Defense or Office of Naval Research. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation hereon.

## References

Amin, M. A., Ahsan, M. S., Maua, J., Ahmed, M., & Nur, K. (2026). Physics-informed self-supervised diagnosis of rotating machinery using latent ODEs and transformer encoders. *PloS one*, *21*, e0339239.

Anthropic (2026). Claude code agent sdk documentation. `https://code.claude.com/docs/en/agent-sdk/overview`. Accessed: 2026-07-22.

Cox, M. T. (2005). Metacognition in computation: A selected research review. *Artificial intelligence*, *169*, 104–141.

Cox, M. T., & Ram, A. (1999). Introspective multistrategy learning: On the construction of learning strategies. *Artificial Intelligence*, *112*, 1–55.

Davis, R. (1984). Diagnostic reasoning based on structure and behavior. *Artificial intelligence*, *24*, 347–410.

Davis, R., & Hamscher, W. (1988). Model-based reasoning: Troubleshooting. In *Exploring artificial intelligence*, 297–346. Elsevier.

De Kleer, J., & Williams, B. C. (1987). Diagnosing multiple faults. *Artificial intelligence*, *32*, 97–130.

Forbus, K. D. (2008). Qualitative modeling. *Foundations of Artificial Intelligence*, *3*, 361–393.

Ghasemi, S. H., Nowak, A. S., & Nazari, R. (2025). Analyzing reliability missteps: The collapse of the francis scott key bridge—target reliability, redundancy, and extreme load combinations.

*Structures* (p. 108590). Elsevier.

Grantham, S. D., & Ungar, L. H. (1990). A first principles approach to automated troubleshooting of chemical plants. *Computers & chemical engineering*, *14*, 783–798.

Heckerman, D., Breese, J. S., & Rommelse, K. (1995). Decision-theoretic troubleshooting. *Communications of the ACM*, *38*, 49–57.

Hopkins, J., Bakler, M., & Khan, A. (2026). Factorio learning environment. *Advances in Neural Information Processing Systems*, *38*.

Kirk, J. R., Wray, R. E., Lindes, P., & Laird, J. E. (2024). Improving Knowledge Extraction from LLMs for Task Learning through Agent Analysis. *Proceedings of the 38th Annual AAAI Conference on Artificial Intelligence*. Vancouver.

Klenk, M., Piotrowski, W., Stern, R., Mohan, S., & de Kleer, J. (2020). Model-based novelty adaptation for open-world ai. *Proceedings of the Eighth Annual Conference on Advances in Cognitive Systems*.

Laird, J., Lebiere, C., Rosenbloom, P., & Stocco, A. (2026). Unified, comprehensive metacognition within the common model of cognition. *International Conference on Artificial General Intelligence* (pp. 1–15). Springer.

Laird, J. E. (2012). *The Soar Cognitive Architecture*. Cambridge, MA: MIT Press.

Laird, J. E., Lebiere, C., & Rosenbloom, P. S. (2017). A standard model of the mind: Toward a common computational framework across artificial intelligence, cognitive science, neuroscience, and robotics. *AI Magazine*, *38*, 13–26.

Lane, D. C. (2008). The emergence and use of diagramming in system dynamics: A critical account. *Systems Research and Behavioral Science*, *25*, 3–23.

Li, Y., Lv, Y., Yan, Y., Gao, X., Jiang, X., & Liu, Y. (2025). Physics-informed neural networks for robust equivalent damping parameter inversion and fault diagnosis in gas-insulated switchgear vibration systems. *Scientific reports*, *15*, 24521.

Musliner, D. J., Pelican, M., McLure, M., Johnston, S., Freedman, R. G., & Knutson, C. (2021). Openmind: planning and adapting in domains with novelty. *Proceedings of the Ninth Annual Conference on Advances in Cognitive Systems*.

Reiter, R. (1987). A theory of diagnosis from first principles. *Artificial intelligence*, *32*, 57–95.

Schoenenberger, L., Schmid, A., Tanase, R., Beck, M., & Schwaninger, M. (2021). Structural analysis of system dynamics models. *Simulation Modelling Practice and Theory*, *110*, 102333.

Venkatasubramanian, V., Rengaswamy, R., Yin, K., & Kavuri, S. N. (2003). A review of process fault detection and diagnosis: Part I: Quantitative model-based methods; Part II: Qualitative models and search engines; Part III: Process history based methods. *Computers & Chemical Engineering*, *27*, 293–346.

Wang, D., Chen, Z., Ni, J., Tong, L., Wang, Z., Fu, Y., & Chen, H. (2023). Interdependent causal networks for root cause localization. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (pp. 5051–5060).

Wube Software Ltd. (2020). Factorio. From `https://www.factorio.com`. Video game.

Zhang, J., Yu, S., Chong, D., Sicilia, A., Tomz, M. R., Manning, C. D., & Shi, W. (2025). Verbalized sampling: How to mitigate mode collapse and unlock LLM diversity. *arXiv preprint arXiv:2510.01171*.

Zilberstein, S. (1996). Using anytime algorithms in intelligent systems. *AI magazine*, *17*, 73–73.